

LAB5_REPORT_PIPELINE CPU

컴퓨터소프트웨어학부 2021026108 김민제

구현 환경

운영체제 : macOS Sonoma

verilog : iverilog

vcd : gtkwave

cpp : Apple clang version 16.0.0 (clang-1600.0.26.6)

지난 과제 파일

ALU.cpp ALU.h ALU.v RF.cpp RF.h RF.v MEM.h MEM.cpp MEM.v 는 single cycle cpu 구현과 동일한 파일을 사용하였습니다.

전체 구조 분석 및 설계

pipeline 구현의 경우 singlecycle처럼 모든 명령어들이 5단계로 돌지만 매 단계 중간에 latch가 존재하여 각 값들을 저장하여 단계별로 노는 단계없이 동작하도록 합니다. 특히 이 때 RAW Hazard가 발생하거나 기본적으로 branch not taken을 기준으로 PC 값을 업데이트하다 branch taken이 발생하게 되면 FLUSH를 일으켜서 잘못 Fetch한 명령어들을 제거해주도록 하여 구현했습니다. 기본적으로 책에 있는 pipeline cpu 설계도를 따라서 만들었고 jal, jr 등의 기능만 따로 추가하여 만들었습니다.

CPP 설계 및 구현

I. LATCH.h

Latch 관리의 용이를 위해서 LATCH 파일을 만들어서 IF_ID, ID_EX, EX_MEM, MEM_WB Latch를 관리했습니다.

II. HAZARD.h & HAZARD.cpp

ID_EX, EX_MEM, MEM_WB Latch의 WB signal의 Regwrite과 wr_addr를 ID의 rs, rt의 값과 비교하고 사용하는지 여부를 확인하여 값(stall 여부)을 반환합니다.

```
if (ID_EX_curr.WB.RegWrite) {
    uint32_t wr_addr = ID_EX_curr.EX.SavePC ? 31 :
        (ID_EX_curr.EX.RegDst ? ID_EX_curr.rd : ID_EX_curr.rt);
    if ((reads_rs && wr_addr == rs_l) || (reads_rt && wr_addr == rt_l)) {
        if (VERBOSE) cout << "[Hazard] RAW hazard on ID/EX stage\n";
        return true;
    }
}
```

III. CPU.cpp

1. cpp의 경우에는 latch에 값을 동시에 업데이트하는 것을 구현하기 어려워 next와 curr로 latch를 2개로 나누어 구현했습니다.
2. 처음에는 hazard detection을 위해서 ID 단계에서 parsing한 명령어들을 보고 rs, rt를 사용하는지 여부를 계산해줍니다.
3. 이후 계산한 결과를 hazard 유닛에게 넘겨서 stall을 계산합니다.
4. 만약 stall이라면 IF, ID 단계는 값을 유지하고 다른 동작을 하지 않도록 합니다.
5. 만약 stall이 아니라면 IF 단계에서는 IF_ID latch 값을 업데이트 해주고, ID 단계에서는 ID_EX 값을 업데이트 해줍니다.
6. EX 단계에서는 alu 연산을 하고 control flow 명령어라면 jump target을 계산해주고 branch_taken을 true로 변경해줍니다. 그리고 바로 flush를 동작하여 IF_ID, ID_EX 를 reset 시킵니다.

```
if (ID_EX_curr.EX.Branch) {
    if (EX_MEM_next.zero) {
```

```

    EX_MEM_next.PC = ID_EX_curr.PC + (ID_EX_curr.ext_imm << 2);
    EX_MEM_next.branch_taken = true;
}
} else if (ID_EX_curr.EX.JR) {
    EX_MEM_next.PC = ID_EX_curr.read_data1;
    EX_MEM_next.branch_taken = true;
} else if (ID_EX_curr.EX.Jump) {
    EX_MEM_next.PC = (ID_EX_curr.PC & 0xF0000000) | (ID_EX_curr.immj << 2);
    EX_MEM_next.branch_taken = true;
}

```

- MEM 단계에서는 EX_MEM의 값으로 memory read나 memory write을 동작하고 MEM_WB 값을 세팅합니다.
- WB 단계에서는 MEM_WB 값을 바탕으로 WB 동작을 합니다.
- 마지막으로 curr = next를 통해서 모든 latch의 값을 반영해주고 PC 값도 업데이트 하여 다음 사이클을 시작합니다.

CPP 동작 확인

makefile에 새로 추가한 HAZARD.cpp를 추가했습니다.

```
SRC = main.cpp CPU.cpp ALU.cpp RF.cpp MEM.cpp CTRL.cpp HAZARD.cpp
```

I. log와 MARS 값 비교

iverilog로 컴파일하여 실행시킨 파일의 결과와 MARS의 결과와 비교한 결과 일치하는 것을 확인했습니다.

\$zero 00000000	\$zero 00000000	\$zero 00000000	\$zero 00000000	\$zero 00000000	\$zero 00000000	\$zero 00000000
\$at ffffa6e4	\$at fff90000	\$at 00000000	\$at c0bf0000	\$at 224d0000	\$at aa210000	\$at e2810000
\$v0 00000000	\$v0 00000005	\$v0 00000028	\$v0 00000000	\$v0 00000005	\$v0 00000005	\$v0 00000000
\$v1 00000000	\$v1 00000000	\$v1 00000000	\$v1 00000000	\$v1 00000000	\$v1 00000000	\$v1 00000000
\$a0 00000000	\$a0 00000000	\$a0 00000000	\$a0 00000000	\$a0 00000000	\$a0 00000000	\$a0 00000000
\$a1 00000000	\$a1 00000000	\$a1 00000000	\$a1 00000000	\$a1 00000000	\$a1 00000000	\$a1 00000000
\$a2 00000000	\$a2 00000000	\$a2 00000000	\$a2 00000000	\$a2 00000000	\$a2 00000000	\$a2 00000000
\$a3 00000000	\$a3 00000000	\$a3 00000000	\$a3 00000000	\$a3 00000000	\$a3 00000000	\$a3 00000000
\$t0 000001db	\$t0 fff8e8df	\$t0 0000030c	\$t0 ccea353c	\$t0 74e250d2	\$t0 d7c716b4	\$t0 ffffffff
\$t1 00000001	\$t1 fff9d354	\$t1 00000028	\$t1 5d5f78a6	\$t1 fbd5a118	\$t1 d7c716b4	\$t1 00000000
\$t2 ffffa6e5	\$t2 fff8e8df	\$t2 00000028	\$t2 b25816d9	\$t2 a16cc4d0	\$t2 00000000	\$t2 ffffffff
\$t3 00000001	\$t3 fff8e8df	\$t3 00000000	\$t3 996e0934	\$t3 5efc37c0	\$t3 00000000	\$t3 ffffffff
\$t4 ffff4a25	\$t4 ffea7884	\$t4 00000000	\$t4 6445d3d1	\$t4 638f9338	\$t4 00003457	\$t4 00000000
\$t5 00000000	\$t5 fff9f0cb	\$t5 00000000	\$t5 d5cd61f4	\$t5 e6442b6e	\$t5 00000000	\$t5 00000000
\$t6 00000000	\$t6 00000000	\$t6 00000000	\$t6 00000000	\$t6 00000000	\$t6 00000000	\$t6 00000000
\$t7 00000000	\$t7 00000000	\$t7 00000000	\$t7 00000000	\$t7 00000000	\$t7 00000000	\$t7 00000000
\$s0 ffffffff	\$s0 fff8e8df	\$s0 00000000	\$s0 c017ab77	\$s0 e219cc86	\$s0 00000000	\$s0 00000000
\$s1 ffffb3a3	\$s1 fff8e8df	\$s1 00000000	\$s1 00000000	\$s1 11bba2a2	\$s1 00000000	\$s1 00000000
\$s2 00000001	\$s2 fff8e8df	\$s2 00000000	\$s2 00000000	\$s2 65a4c2ac	\$s2 00000000	\$s2 00000000
\$s3 ffff4221	\$s3 fff9d354	\$s3 00000000	\$s3 00000000	\$s3 13d0e99e	\$s3 00000000	\$s3 00000000
\$s4 00000000	\$s4 ffea7884	\$s4 00000000	\$s4 00000000	\$s4 61116734	\$s4 00000000	\$s4 00000000
\$s5 00000001	\$s5 00042808	\$s5 00000000	\$s5 00000000	\$s5 fdead08c	\$s5 00000000	\$s5 00000000
\$s6 00000000	\$s6 00000000	\$s6 00000000	\$s6 00000000	\$s6 00000000	\$s6 00000000	\$s6 00000000
\$s7 00000000	\$s7 00000000	\$s7 00000000	\$s7 00000000	\$s7 00000000	\$s7 00000000	\$s7 00000000
\$t8 00000001	\$t8 00000000	\$t8 00000000	\$t8 00000000	\$t8 00000000	\$t8 00000000	\$t8 00000000
\$t9 00000596	\$t9 00000000	\$t9 00000000	\$t9 00000000	\$t9 00000000	\$t9 00000000	\$t9 00000000
\$k0 00000000	\$k0 00000000	\$k0 00000000	\$k0 00000000	\$k0 00000000	\$k0 00000000	\$k0 00000000
\$k1 00000000	\$k1 00000000	\$k1 00000000	\$k1 00000000	\$k1 00000000	\$k1 00000000	\$k1 00000000
\$gp 00001800	\$gp 00001800	\$gp 00001800	\$gp 00001800	\$gp 00001800	\$gp 00001800	\$gp 00001800
\$sp 00003ffc	\$sp 00003ffc	\$sp 00003ffc	\$sp 00003ffc	\$sp 00003ffc	\$sp 00003ffc	\$sp 00003ffc
\$fp 00000000	\$fp 00000000	\$fp 00000000	\$fp 00000000	\$fp 00000000	\$fp 00000000	\$fp 00000000
\$ra 00000000	\$ra 00000000	\$ra 00000000	\$ra 00000000	\$ra 00000034	\$ra 00000000	\$ra 00000000

Verilog 설계 및 구현

I. GLOBAL.v, ALU.v, MEM.v, CTRL.v

HW3의 single cycle cpu에서 사용한 파일을 그대로 사용했습니다.

```

`define STATE_IF      4'd1
`define STATE_ID      4'd2
`define STATE_EX      4'd3
`define STATE_MEM     4'd4
`define STATE_WB      4'd5
`define STATE_ERROR   4'd6

```

II. RF.v

RF에서 stall을 줄이기 위해서 WB stage 에선 Internal forwarding을 사용하도록 했습니다.

```
always @(*) begin
    if (RegWrite && wr_addr != 0 && wr_addr == rd_addr1)
        rd_data1 = wr_data;
    else
        rd_data1 = register_file[rd_addr1];

    if (RegWrite && wr_addr != 0 && wr_addr == rd_addr2)
        rd_data2 = wr_data;
    else
        rd_data2 = register_file[rd_addr2];
end
```

III. HAZARD.v

1. cpp에서는 main에서 use_rs, use_rt를 계산했지만 verilog에서는 타이밍 이슈를 해결하기 위해서 HAZARD 유닛 내부에서 계산하도록 했습니다.
2. ID 단계의 명령어가 0이 아니라면 cpp과 동일하게 hazard를 판별하여 stall을 output시킵니다.

```
always @(*) begin
    if (inst != 32'd0) begin
        if (EX_RegWrite) begin
            if (((use_rs && (ex_wr_addr == rs_l)) || (use_rt && (ex_wr_addr == rt_l)))) begin
                stall = 1'b1;
            end
        end
    end
    else if (MEM_RegWrite &&
        ((use_rs && (MEM_wr_addr == rs_l)) || (use_rt && (MEM_wr_addr == rt_l)))) begin
        stall = 1'b1;
    end
    else if (WB_RegWrite &&
        ((use_rs && (WB_wr_addr == rs_l)) || (use_rt && (WB_wr_addr == rt_l)))) begin
        stall = 1'b1;
    end
    else begin
        stall = 1'b0;
    end
    end else begin
        stall = 1'b0;
    end
end
```

III. CPU.v

1. cpp에서는 latch.h를 만들어서 latch를 따로 만들었습니다. verilog에서도 latch.v를 만들어서 따로 latch를 관리하려 했지만 구현에 있어서 너무 많은 input과 output을 적어야해 오히려 더 불편한 점이 많아서 그냥 CPU.v에 latch를 reg로 구현했습니다.
2. cpp에서는 curr, next latch를 나누어 따로 관리를 했지만 verilog에서는 NONBLOCKING을 지원하므로 그냥 하나의 latch만을 이용해서 동기로 값을 변경시켜주었습니다.
3. Hazard 유닛으로 hazard를 계산하여 stall을 걸어주고 각 단계의 latch들에 동기로 값을 쓰도록 해서 그 쓰인 값들로 RF, ALU, MEM 유닛에 input을 넣어주어 동작하도록 했습니다.

4. 특히 저번 과제부터 PC를 업데이트 하는 과정에서 PC_next에 값을 저장했다가 사용하면 사이클이 어긋나는 문제가 있어서 cpp과 다르게 그냥 즉시 PC에 값을 쓰도록 했습니다. 이렇게 구현해야 cpp 처럼 바로 PC가 다음 사이클에 반영되 것과 동일 하기 때문입니다.
5. 이 외의 부분들은 cpp 구현을 거의 동일하게 verilog에 옮겨서 만들었습니다.

```

if(branch_taken) begin
    branch_taken <= 1'b0;
end else begin
    if (ID_EX_EX_Branch) begin
        if(alu_result) begin
            EX_MEM_PC <= ID_EX_PC + (ID_EX_ext_imm << 2);
            branch_taken <= 1'b1;
        end else begin
            EX_MEM_PC <= 0;
            branch_taken <= 1'b0;
        end
    end else if (ID_EX_EX_JR) begin
        EX_MEM_PC <= ID_EX_read_data1;
        branch_taken <= 1'b1;
    end else if (ID_EX_EX_Jump) begin
        EX_MEM_PC <= {ID_EX_PC[31:28], ID_EX_immj, 2'b00};
        branch_taken <= 1'b1;
    end else begin
        EX_MEM_PC <= 0;
        branch_taken <= 1'b0;
    end
end
end

```

Verilog 동작 확인

아래 명령어로 컴파일하여 cpu 파일을 실행하여 동작 결과를 확인했습니다.

```
iverilog -o cpu ALU.v CPU.v CPU_tb.v CTRL.v GLOBAL.v MEM.v RF.v HAZARD.v
```

확인 결과 모두 SUCCESS가 뜨며 올바르게 동작하는 것을 확인할 수 있었습니다.

```

> ./cpu
VCD info: dumpfile cpu.vcd opened for output.
Program Terminate

Simulation success!!!
CPU_tb.v:58: $finish called at 2619935000 (1ps)

```

동작 시간이나 이런 부분은 종료조건을 단순히 매우 큰 PC 값으로 설정해두어서 큰 의미가 없어서 따로 첨부하지 않았습니다.

Trouble Shooting

I. 종료 조건에 대한 문제

single cycle 이후부터 계속해서 IF 단계에서 inst를 0을 읽으면 바로 종료가 되어 ID, EX, MEM, WB 단계에서 처리되고 있던 명령어들이 올바르게 종료되지 못하는 문제가 있었습니다. 그래서 따로 종료 조건을 만들어 주어야했는데 생각한 방법으론 WB을 할 때 0에 값을 쓰려고 하는 경우가 있다면 종료시키거나 instruction 자체를 latch에 넘겨서 WB 단계에서 0인 instruction을 받으면 종료하도록 하는 방법을 생각했습니다. 하지만 실제 CPU에서는 이러한 특정한 조건으로 cpu를 종료시는 것은 이상하다고 생각했습니다. 그래서 그냥 mem에서 overflow가 나면 스스로 종료를 하고 0인 명령어는 어떠한 영향도 주지 않기 때문에 그냥 따로 종

료조건을 추가하지 않았습니다. 단순히 instruction memory bound를 넘으면 끝나도록 했습니다. 이것을 설정할 때 bound 값을 너무 크게 설정했더니 testcase #4 에서 sw를 하여 dmem에 적힌 값을 명령어로 읽어서 동작하는 경우가 있어서 수정했습니다.

II. CPP 구현에서 FLUSH timing 관련 문제

처음에는 cpp에서 branch_taken이 일어나면 Flush를 다음 사이클에 일어나도록 하여 사이클이 한 사이클 씩 밀리고 제대로 동작하지 않는 문제가 있었습니다. 그래서 그냥 바로 Flush가 일어나면 즉시 해결하도록 했습니다.

하지만 이렇게 구현하니 verilog에서는 latch의 값을 비동기로 즉시 flush가 일어나면 처리하는 것이 불가능하여 한 cycle은 flush를 처리하는데 소비하도록 하였습니다. 성능을 향상시키기 위해서 바로 IF 단계에서는 값을 지우면서 instruction을 Fetch 해올 수도 있을 것이라 생각은 하였지만 따로 구현하지는 않았습니다.

III. Verilog testcase #2 → Branch Taken 시 zero 값 계산 문제

jump가 일어나는 상황이 EX 단계에서 바로 계산되어 넘어가는 것이어서 zero 값이 계산되고 나서 그 값을 저장하여 다음 사이클에 사용되는 형태여서 verilog에서 branch taken이 이상하게 발생하는 문제가 있었습니다. cpp에서는 바로 다음 latch에 들어갈 데이터의 끝을 다 써서 가능했는데 이 부분이 문제가 되어 그냥 따로 branch taken을 확인해주는 유닛(따로 만들지는 않고 그냥 계산만 하는 식으로 했습니다)을 만들어서 직접 read_data1, read_data2를 비교하여 zero 값을 대신 연산하도록 하였습니다.

⇒ 위 방식으로 구현을 하고 코드를 돌아보며 분석하다보니 그렇게 된다면 branch 연산에서는 alu 연산을 하지 않는 것처럼 동작하고 저것 만을 위한 또 다른 유닛이 필요한거라 구현상에서 이상하다고 생각해서 다시 cpp 코드를 뜯어보니 next를 사용하기는 하지만 그 값이 zero(alu_result) 값을 쓰는 것이어서 그냥 alu_result를 쓰도록 하여 해결했습니다.

IV. Verilog testcase #5 → JAL 이후 addu 연산 문제

체크 표시한 명령어가 동작할 때 이상한 값이 쓰였습니다. 이 시점에선 테스트 케이스 5번을 제외하고는 모드 잘 통과하고 있는 상황 이어서 addu 명령어가 연속적으로 있는 상황에서 도대체 무슨 에러가 발생하는지 찾기가 매우 어려웠습니다. 위치상에 있는게 12번 레지스터의 값을 가져오는데 이전의 값을 가져오는 것을 찾았고 internal forwarding을 적용하여 이를 해결했습니다.

☐	0x000002a8	0x01086021 addu \$12,\$8,\$8	162:	addu \$t4, \$t0, \$t0
☐	0x000002ac	0x01096021 addu \$12,\$8,\$9	163:	addu \$t4, \$t0, \$t1
☐	0x000002b0	0x010a9021 addu \$18,\$8,\$10	164:	addu \$s2, \$t0, \$t2
☐	0x000002b4	0x010b9021 addu \$18,\$8,\$11	165:	addu \$s2, \$t0, \$t3
☒	0x000002b8	0x010c9821 addu \$19,\$8,\$12	166:	addu \$s3, \$t0, \$t4
☐	0x000002bc	0x010da021 addu \$20,\$8,\$13	167:	addu \$s4, \$t0, \$t5

그렇다면 왜 cpp에서는 internal forwarding을 적용하지 않았는데 해결이 되었나? 이것은 아마 cpp과 verilog의 차이에서 비롯된 것 같은데 verilog에서 assign 즉 비동기로 동작하는 부분과 동기로 동작하는 부분의 차이가 cpp과는 다르게 동작하는 부분이 많아서 cpp에서는 올바르게 동작했지만 verilog에서는 올바르게 동작하지 못했던 것 같습니다.